

Introdução aos Algoritmos
Terceira edição

Introdução

Esta parte fará com que você comece a pensar sobre como projetar e analisar algoritmos. Isto é pretende ser uma introdução suave sobre como especificamos algoritmos, alguns dos estratégias de design que usaremos ao longo deste livro, e muitas das estratégias fundamentais ideias usadas na análise de algoritmos. As partes posteriores deste livro desenvolverão essa base.

O Capítulo 1 fornece uma visão geral dos algoritmos e seu lugar na comunicação moderna sistemas de colocação. Este capítulo define o que é um algoritmo e lista alguns exemplos. Também defende que devemos considerar algoritmos como uma tecnologia, juntamente tecnologias secundárias, como hardware rápido, interfaces gráficas de usuário, orientação a objetos sistemas e redes.

No Capítulo 2, vemos nossos primeiros algoritmos, que resolvem o problema de classificação uma sequência de n números. Eles são escritos em um pseudocódigo que, embora não diretamente traduzível para qualquer linguagem de programação convencional, transmite a estrutura estrutura do algoritmo de forma clara o suficiente para que você seja capaz de implementá-lo no idioma de sua escolha. Os algoritmos de classificação que examinamos são classificação por inserção, que usa uma abordagem incremental e merge sort, que usa uma tecnologia recursiva nique conhecido como "dividir para conquistar". Embora o tempo de cada um exija aumentos com o valor de n , a taxa de aumento difere entre os dois algoritmos. Nós determinar esses tempos de execução no Capítulo 2, e desenvolvemos uma notação útil para expressá-los.

O Capítulo 3 define precisamente essa notação, que chamamos de notação assintótica. isto começa definindo várias notações assintóticas, que usamos para delimitar algo tempos de execução do ritmo acima e / ou abaixo. O resto do Capítulo 3 é principalmente

uma apresentação de notação matemática, mais para garantir que seu uso de notação corresponde a isso neste livro do que ensinar novos conceitos matemáticos.

O Capítulo 4 aprofunda-se no método de dividir e conquistar introduzido em Capítulo 2. Ele fornece exemplos adicionais de algoritmos de divisão e conquista, incluindo incluindo o método surpreendente de Strassen para multiplicar duas matrizes quadradas. Indivíduo-ter 4 contém métodos para resolver recorrências, que são úteis para descrever os tempos de execução de algoritmos recursivos. Uma técnica poderosa é o “mas-tem método”, que costumamos usar para resolver recorrências que surgem de dividir e conquistar algoritmos. Embora muito do Capítulo 4 seja dedicado a provar o corretidão do método mestre, você pode pular esta prova, mas ainda assim empregar o método mestre método.

O Capítulo 5 apresenta a análise probabilística e algoritmos aleatórios. Nós tip-Use a análise probabilística para determinar o tempo de execução de um algoritmo em casos em que, devido à presença de uma distribuição de probabilidade inerente, o o tempo de execução pode diferir em diferentes entradas do mesmo tamanho. Em alguns casos, nós suponha que as entradas estejam em conformidade com uma distribuição de probabilidade conhecida, de modo que estamos calculando a média do tempo de execução de todas as entradas possíveis. Em outros casos, a probabilidade distribuição não vem das entradas, mas de escolhas aleatórias feitas durante o curso do algoritmo. Um algoritmo cujo comportamento é determinado não apenas por seu entrada, mas pelos valores produzidos por um gerador de números aleatórios é um algoritmo. Podemos usar algoritmos aleatórios para impor uma distribuição de probabilidade nas entradas - garantindo assim que nenhuma entrada em particular sempre causa desempenho ruim mance - ou mesmo para limitar a taxa de erro de algoritmos que podem produzir resultados incorretos em uma base limitada.

Os apêndices A - D contém outro material matemático que você achará útil enquanto você lê este livro. É provável que você tenha visto muito do material no apêndice capítulos antes de ler este livro (embora as definições específicas e as convenções de notação que usamos podem diferir em alguns casos daquilo que você tem visto no passado), então você deve pensar nos Apêndices como material de referência. Por outro lado, você provavelmente ainda não viu a maior parte do material em Parte I. Todos os capítulos da Parte I e os Apêndices são escritos com um tutorial sabor.

1 O Papel dos Algoritmos na Computação

O que são algoritmos? Por que vale a pena estudar algoritmos? Qual é o papel de algoritmos em relação a outras tecnologias usadas em computadores? Neste capítulo, nós vai responder a essas perguntas.

1.1 Algoritmos

Informalmente, um **algoritmo** é qualquer procedimento computacional bem definido que leva algum valor, ou conjunto de valores, como **entrada** e produz algum valor, ou conjunto de valores, como **saída**. Um algoritmo é, portanto, uma sequência de etapas computacionais que transformam a entrada na saída.

Também podemos ver um algoritmo como uma ferramenta para resolver um well-specified **calculation problema internacional**. A declaração do problema especifica em termos gerais o desejado relação de entrada / saída. O algoritmo descreve um procedimento computacional específico último para alcançar essa relação de entrada / saída.

Por exemplo, podemos precisar classificar uma sequência de números em não decrescente ordem. Este problema surge com frequência na prática e fornece um terreno fértil para introduzindo muitas técnicas de design padrão e ferramentas de análise. Aqui é como nós define formalmente o **problema de classificação**:

Entrada: Uma sequência de n números $a_1; a_2; \dots; a_n$.

Resultado: A permutação (reordenamento) $a_{i_1}; a_{i_2}; \dots; a_{i_n}$ para a sequência de entrada, tal que $a_{i_1} \leq a_{i_2} \leq \dots \leq a_{i_n}$.

Por exemplo, dada a sequência de entrada $h31; 41; 59; 26; 41; 58i$, um algoritmo de classificação retorna como saída a sequência $h26; 31; 41; 41; 58; 59i$. Essa sequência de entrada é chamado de **instância** do problema de classificação. Em geral, uma **instância de um problema** consiste na entrada (satisfazendo quaisquer restrições impostas no problema declaração) necessária para calcular uma solução para o problema.

Como muitos programas o usam como uma etapa intermediária, a classificação é fundamental operação em ciência da computação. Como resultado, temos um grande número de boas classificações algoritmos à nossa disposição. Qual algoritmo é melhor para um determinado aplicativo depende em - entre outros fatores - o número de itens a serem classificados, até que ponto os itens já estão classificados de alguma forma, possíveis restrições nos valores dos itens, a arquitetura do computador e o tipo de dispositivos de armazenamento a serem usados: principal memória, discos ou mesmo fitas.

Um algoritmo é considerado **correto** se, para cada instância de entrada, ele para com a saída correta. Dizemos que um algoritmo **resolve** o dado computacional problema. Um algoritmo incorreto pode não parar em algumas instâncias de entrada ou pode parar com uma resposta incorreta. Ao contrário do que você poderia esperar, incorreto algoritmos às vezes podem ser úteis, se pudermos controlar sua taxa de erro. Veremos um exemplo de um algoritmo com uma taxa de erro controlável no Capítulo 31, quando estudar algoritmos para encontrar grandes números primos. Normalmente, no entanto, devemos preocupe-se apenas com os algoritmos corretos.

Um algoritmo pode ser especificado em inglês, como um programa de computador, ou mesmo como um design de hardware. O único requisito é que a especificação deve fornecer um precisa de descrição do procedimento computacional a ser seguido.

Que tipos de problemas são resolvidos por algoritmos?

A classificação não é de forma alguma o único problema computacional para o qual os algoritmos têm foi desenvolvido. (Você provavelmente já suspeitou disso quando viu o tamanho deste livro.) Aplicações práticas de algoritmos são onipresentes e incluem o seguinte exemplos de ing:

O Projeto Genoma Humano fez um grande progresso em direção aos objetivos da identidade identificando todos os 100.000 genes do DNA humano, determinando as sequências do 3 bilhões de pares de bases químicas que constituem o DNA humano, armazenando esta informação em bases de dados e desenvolvimento de ferramentas para análise de dados. Cada uma dessas etapas requer algoritmos sofisticados. Embora as soluções para os vários problemas Os itens envolvidos estão além do escopo deste livro, muitos métodos para resolver esses problemas biológicos usam idéias de vários dos capítulos deste livro, assim permitindo que os cientistas realizem tarefas enquanto usam os recursos de maneira eficiente o a economia está no tempo, tanto humano quanto na máquina, e dinheiro, conforme mais informações ção pode ser extraída de técnicas de laboratório.

A Internet permite que pessoas em todo o mundo acessem e recuperem rapidamente grandes quantidades de informações. Com a ajuda de algoritmos inteligentes, sites no A Internet é capaz de gerenciar e manipular esse grande volume de dados. Exemplos de problemas que fazem uso essencial de algoritmos incluem encontrar boas rotas em que os dados viajarão (técnicas para resolver tais problemas aparecem em

Capítulo 24), e usando um mecanismo de pesquisa para encontrar rapidamente páginas em que as informações residem (as técnicas relacionadas estão nos Capítulos 11 e 32).

O comércio eletrônico permite que bens e serviços sejam negociados e ex- alterado eletronicamente e depende da privacidade das informações pessoais como números de cartão de crédito, senhas e extratos bancários. O núcleo tecnologias usadas no comércio eletrônico incluem criptografia de chave pública e assinaturas digitais (abordadas no Capítulo 31), que são baseadas em algo numérico ritmos e teoria dos números.

Fabricação e outras empresas comerciais muitas vezes precisam alocar escassos recursos da forma mais benéfica. Uma empresa de petróleo pode querer saber onde colocar seus poços de forma a maximizar o lucro esperado. O candidato político pode querer determinar onde gastar dinheiro comprando publicidade de campanha em a fim de maximizar as chances de ganhar uma eleição. Uma companhia aérea pode desejar atribuir tripulações aos voos da maneira menos cara possível, certificando-se de que cada voo é coberto e as regulamentações governamentais em relação à programação da tripulação são atendidos. Um provedor de serviços de Internet pode desejar determinar onde colocar recursos adicionais para atender seus clientes de forma mais eficaz. Tudo de estes são exemplos de problemas que podem ser resolvidos usando programação linear, que estudaremos no Capítulo 29.

Embora alguns dos detalhes desses exemplos estejam além do escopo deste livro, forneceremos técnicas subjacentes que se aplicam a esses problemas e problemas áreas. Também mostramos como resolver muitos problemas específicos, incluindo o seguinte:

Recebemos um roteiro no qual a distância entre cada par de cruzamentos são marcados, e desejamos determinar o trajeto mais curto de um cruzamento para outro. O número de rotas possíveis pode ser enorme, mesmo se nós não permitir rotas que se cruzem. Como escolhemos qual de todos rotas possíveis é o mais curto? Aqui, modelamos o roteiro (que é ele mesmo um modelo das estradas reais) como um gráfico (que encontraremos na Parte VI e Apêndice B), e queremos encontrar o caminho mais curto de um vértice para outro no gráfico. Veremos como resolver esse problema de forma eficiente no Capítulo 24.

Recebemos duas sequências ordenadas de símbolos, $X = x_1; x_2; \dots; x_m$ e $Y = y_1; y_2; \dots; y_n$, e desejamos encontrar uma subsequência comum mais longa de X e Y . Uma subsequência de X é apenas X com alguns (ou possivelmente todos ou nenhum) de seus elementos removidos. Por exemplo, uma subsequência de $hA; B; C; D; E; F; G$ seria $hB; C; E; G$. O comprimento de uma subsequência comum mais longa de X e Y dá uma medida de quão semelhantes essas duas sequências são. Por exemplo, se as duas sequências são pares de bases em fitas de DNA, então podemos considerar eles são semelhantes se eles têm uma longa subsequência comum. Se X tiver m símbolos e Y tem n símbolos, então X e Y têm 2^m e 2^n possíveis subsequências,

respectivamente. Seleccionando todas as subsequências possíveis de X e Y e combinando eles podem levar um tempo proibitivamente longo, a menos que m e n sejam muito pequenos. Veremos no Capítulo 15 como usar uma técnica geral conhecida como dinâmica programação para resolver este problema com muito mais eficiência.

Recebemos um projeto mecânico em termos de uma biblioteca de peças, onde cada peça pode incluir instâncias de outras peças, e precisamos listar as peças para que que cada parte aparece antes de qualquer parte que a usa. Se o design compreender n partes, então há $n!$ ordens possíveis, onde $n!$ indica a função fatorial. Como a função fatorial cresce mais rápido do que até mesmo uma função exponencial, não podemos gerar de forma viável cada pedido possível e, em seguida, verificar se, dentro essa ordem, cada parte aparece antes das partes que a usam (a menos que tenhamos apenas algumas partes). Este problema é uma instância de classificação topológica, e veremos no Capítulo 22, como resolver esse problema com eficiência.

Recebemos n pontos no plano, e desejamos encontrar o casco convexo de esses pontos. O casco convexo é o menor polígono convexo que contém o pontos. Intuitivamente, podemos pensar em cada ponto como sendo representado por um prego saindo de uma placa. O casco convexo seria representado por um elástico que envolve todas as unhas. Cada prego em torno do qual a borracha

banda faz uma volta é um vértice do casco convexo. (Veja a Figura 33.6 na página 1029 por exemplo.) Qualquer um dos 2 n subconjuntos dos pontos podem ser os vértices do casco convexo. Saber quais pontos são vértices do casco convexo é não o bastante, também, uma vez que também precisamos saber a ordem em que eles aparecer. Existem muitas opções, portanto, para os vértices do casco convexo. O Capítulo 33 fornece dois bons métodos para localizar o casco convexo.

Estas listas estão longe de ser exaustivas (como você provavelmente imaginou peso deste livro), mas exibem duas características que são comuns a muitos interesses problemas algorítmicos:

1. Eles têm muitas soluções candidatas, a grande maioria das quais não resolve o problema em questão. Encontrar um que funcione ou que seja o "melhor" pode apresentar um grande desafio.
2. Eles têm aplicações práticas. Dos problemas na lista acima, encontrar o caminho mais curto fornece os exemplos mais fáceis. Uma empresa de transporte, como uma empresa de transporte rodoviário ou ferroviário, tem interesse financeiro em encontrar os caminhos mais curtos através de uma rede rodoviária ou ferroviária, porque tomar caminhos mais curtos resulta em menor custos de mão de obra e combustível. Ou um nó de roteamento na Internet pode precisar encontrar o caminho mais curto pela rede para encaminhar uma mensagem rapidamente. Agora pessoa que deseja dirigir de Nova York a Boston pode querer encontrar um carro instruções de um site apropriado, ou ela pode usar seu GPS enquanto dirige.

Nem todo problema resolvido por algoritmos tem um conjunto facilmente identificado de candidatos soluções. Por exemplo, suponha que recebamos um conjunto de valores numéricos representando amostras de um sinal, e queremos calcular a transformada discreta de Fourier de essas amostras. A transformada discreta de Fourier converte o domínio do tempo para o domínio de frequência, produzindo um conjunto de coeficientes numéricos, para que possamos determinar a força de várias frequências no sinal amostrado. Além de mentir em o coração do processamento de sinal, as transformações discretas de Fourier têm aplicações em dados compressão e multiplicação de grandes polinômios e inteiros. Capítulo 30 dá um algoritmo eficiente, a transformada rápida de Fourier (comumente chamada de FFT), para este problema, e o capítulo também esboça o projeto de um circuito de hardware para calcule o FFT.

Estruturas de dados

Este livro também contém várias estruturas de dados. Uma **estrutura de dados** é uma forma de armazenar e organizar os dados para facilitar o acesso e modificações. Sem dados únicos estrutura funciona bem para todos os fins, por isso é importante conhecer os pontos fortes e limitações de vários deles.

Técnica

Embora você possa usar este livro como um "livro de receitas" para algoritmos, você pode algum dia encontrar um problema para o qual você não pode encontrar prontamente um algoritmo publicado (muitos dos exercícios e problemas deste livro, por exemplo). Este livro vai te ensinar técnicas de design e análise de algoritmos para que você possa desenvolver algoritmos em o seu, mostre que eles dão a resposta correta e entenda sua eficiência. Capítulos diferentes abordam diferentes aspectos da resolução de problemas algorítmicos. Alguns capítulos abordam problemas específicos, como encontrar medianas e estatísticas de ordem em Capítulo 9, computando árvores geradoras mínimas no Capítulo 23, e determinando um fluxo máximo em uma rede no Capítulo 26. Outros capítulos abordam técnicas, como dividir e conquistar no Capítulo 4, programação dinâmica no Capítulo 15, e análise amortizada no Capítulo 17.

Problemas difíceis

A maior parte deste livro é sobre algoritmos eficientes. Nossa medida usual de eficiência é a velocidade, ou seja, quanto tempo um algoritmo leva para produzir seu resultado. Há alguns problemas, entretanto, para os quais nenhuma solução eficiente é conhecida. Capítulo 34 estudos um subconjunto interessante desses problemas, que são conhecidos como NP-completos.

Por que os problemas NP-completos são interessantes? Primeiro, embora não seja eficiente ritmo para um problema NP-completo já foi encontrado, ninguém jamais provou

que um algoritmo eficiente para um não pode existir. Em outras palavras, ninguém sabe se existem ou não algoritmos eficientes para problemas NP-completos. Segundo, o conjunto de problemas NP-completos tem a propriedade notável de que se um algo eficiente O ritmo existe para qualquer um deles, então algoritmos eficientes existem para todos eles. esta relação entre os problemas NP-completos torna a falta de soluções eficientes ainda mais tentador. Terceiro, vários problemas NP-completos são semelhantes, mas não idênticos, a problemas para os quais conhecemos algoritmos eficientes. Computador os cientistas estão intrigados em como uma pequena mudança na definição do problema pode causar uma grande mudança na eficiência do algoritmo mais conhecido.

Você deve saber sobre os problemas NP-completos porque alguns deles surgem surpreendentemente frequentemente em aplicações reais. Se você for chamado para produzir um eficiente algoritmo para um problema NP-completo, é provável que você gaste muito tempo em uma busca infrutífera. Se você puder mostrar que o problema é NP-completo, você pode, em vez disso, gaste seu tempo desenvolvendo um algoritmo eficiente que fornece uma boa, mas não a melhor solução possível.

Como um exemplo concreto, considere uma empresa de entrega com um depósito central. Cada dia, ele carrega cada caminhão de entrega no depósito e os envia para entregar as mercadorias para vários endereços. No final do dia, cada caminhão deve voltar ao depósito para que esteja pronto para ser carregado no dia seguinte. Para reduzir custos, a empresa quer para selecionar uma ordem de paradas de entrega que produza a menor distância geral percorrida por cada caminhão. Este é o conhecido "problema do caixeiro viajante" e é NP-completo. Não tem algoritmo eficiente conhecido. Sob certas suposições, no entanto, sabemos de algoritmos eficientes que fornecem uma distância geral que é não muito acima do menor possível. O Capítulo 35 discute essa "aproximação algoritmos."

Paralelismo

Por muitos anos, pudemos contar com o aumento da velocidade do clock do processador em um ritmo constante taxa. Limitações físicas representam um obstáculo fundamental para um relógio sempre crescente velocidades, no entanto: porque a densidade de potência aumenta superlinearmente com a velocidade do clock, os chips correm o risco de derreter quando suas velocidades de clock se tornam altas o suficiente. Em ordem para realizar mais cálculos por segundo, portanto, os chips estão sendo projetados para contêm não apenas uma, mas várias "cores" de processamento. Podemos comparar esses multicore computadores para vários computadores sequenciais em um único chip; em outras palavras, eles são um tipo de "computador paralelo". A fim de obter o melhor desempenho de vários núcleos computadores, precisamos projetar algoritmos com o paralelismo em mente. Capítulo 27 apresenta um modelo para algoritmos "multithreaded", que tiram proveito de vários Cores. Este modelo tem vantagens do ponto de vista teórico, e forma a base de vários programas de computador de sucesso, incluindo um campeonato de xadrez programa.

Exercícios

1.1-1

Dê um exemplo do mundo real que requer classificação ou um exemplo do mundo real que requer computando um casco convexo.

1.1-2

Além da velocidade, quais outras medidas de eficiência podem ser usadas em um mundo real configuração?

1.1-3

Selecione uma estrutura de dados que você viu anteriormente e discuta seus pontos fortes e limitações.

1.1-4

Como os problemas do caminho mais curto e do caixeiro viajante dados acima são semelhantes? Como eles são diferentes?

1.1-5

Encontre um problema do mundo real em que apenas a melhor solução funcione. Então chegar a um em que uma solução que seja "aproximadamente" a melhor seja boa o suficiente.

Suponha que os computadores sejam infinitamente rápidos e a memória do computador esteja livre. Seria você tem algum motivo para estudar algoritmos? A resposta é sim, mesmo que não seja por outro motivo do que você ainda gostaria de demonstrar que seu método de solução termina e faz isso com a resposta correta.

Se os computadores fossem infinitamente rápidos, qualquer método correto para resolver um problema faria. Você provavelmente gostaria que sua implementação estivesse dentro dos limites de boas práticas de engenharia de software (por exemplo, sua implementação deve ser bem projetado e documentado), mas você usaria mais frequentemente qualquer método foi o mais fácil de implementar.

Claro, os computadores podem ser rápidos, mas não são infinitamente rápidos. E memória pode ser barato, mas não é grátis. O tempo de computação é, portanto, um limite recurso, e também o espaço na memória. Você deve usar esses recursos com sabedoria e algoritmos eficientes em termos de tempo ou espaço o ajudarão a fazer isso.

Eficiência

Diferentes algoritmos concebidos para resolver o mesmo problema muitas vezes diferem dramaticamente em sua eficiência. Essas diferenças podem ser muito mais significativas do que diferenças devido ao hardware e software.

Como exemplo, no Capítulo 2, veremos dois algoritmos de classificação. O primeiro, conhecido como **classificação por inserção**, leva tempo aproximadamente igual a $c_1 n^2$ para classificar n itens, onde c_1 é uma constante que não depende de n . Ou seja, leva um tempo aproximadamente proporcional para n^2 . O segundo, **merge sort**, leva tempo aproximadamente igual a $c_2 n \lg n$, onde $\lg n$ representa $\log_2 n$ e c_2 é outra constante que também não depende de n . Inserir a ordenação por comparação normalmente tem um fator constante menor do que a ordenação por mesclagem, de modo que $c_1 < c_2$. Veremos que os fatores constantes podem ter muito menos impacto sobre o funcionamento tempo do que a dependência do tamanho de entrada n . Vamos escrever o tipo de inserção em execução tempo como $c_1 n^2$ e tempo de execução do merge sort como $c_2 n \lg n$. Então vemos que onde a classificação por inserção tem um fator de n em seu tempo de execução, a classificação por mesclagem tem um fator de $\lg n$, que é muito menor. (Por exemplo, quando $n = 1000$, $\lg n$ é aproximadamente 10, e quando n é igual a um milhão, $\lg n$ é aproximadamente apenas 20.) Embora a inserção classificar geralmente é executado mais rápido do que mesclar classificação para tamanhos de entrada pequenos, uma vez que o tamanho de entrada torna-se grande o suficiente, a vantagem de merge sort de $\lg n$ vs. n será mais do que compensar pela diferença em fatores constantes. Não importa como muito menor c_1 é do que c_2 , sempre haverá um ponto de cruzamento além do qual a classificação por mesclagem é mais rápida.

Para um exemplo concreto, vamos colocar um computador mais rápido (computador A) executando insert-classificação em relação a um computador mais lento (computador B) executando a classificação de mesclagem. Cada um deles deve classificar uma matriz de 10 milhões de números. (Embora 10 milhões de números possam parecer muito, se os números são inteiros de oito bytes, então a entrada ocupa cerca de 80 megabytes, que cabem na memória até de um laptop barato. computador muitas vezes.) Suponha que o computador A execute 10 bilhões de instruções por segundo (mais rápido do que qualquer computador sequencial único no momento da redação deste artigo) e o computador B executa apenas 10 milhões de instruções por segundo, de modo que o computador A é 1000 vezes mais rápido que o computador B em termos de capacidade de computação bruta. Fazer a diferença ainda mais dramática, suponha que o programador mais astuto do mundo classificação de inserção de códigos em linguagem de máquina para o computador A, e o código resultante requer $2n$ instructions: para classificar n números. Suponha ainda que apenas uma média programador implementa merge sort, usando uma linguagem de alto nível com um ineficiente compilador, com o código resultante levando $50n \lg n$ instruções. Para classificar 10 milhões números, o computador A leva

$2 \cdot 10^7 / 2$ instruções
 10^{10} instruções / segundo

D 20.000 segundos (mais de 5,5 horas);

enquanto o computador B leva

50 10 γ lg10 γ instruções 1163 segundos (menos de 20 minutos):
10 γ instruções / segundo

Ao usar um algoritmo cujo tempo de execução cresce mais lentamente, mesmo com um compilador, o computador B roda mais de 17 vezes mais rápido que o computador A! O adv-
O tipo de mesclagem é ainda mais pronunciado quando classificamos 100 milhões de números:
onde a classificação por inserção leva mais de 23 dias, a classificação por mesclagem leva menos de quatro horas.
Em geral, conforme o tamanho do problema aumenta, também aumenta a vantagem relativa de mesclagem ordenar.

Algoritmos e outras tecnologias

O exemplo acima mostra que devemos considerar algoritmos, como hardwares de computador ware, como uma **tecnologia**. O desempenho total do sistema depende da escolha eficiente algoritmos tanto quanto na escolha de hardware rápido. Assim como os avanços rápidos estão sendo feitos em outras tecnologias de computador, eles estão sendo feitos em algoritmos também.

Você pode se perguntar se os algoritmos são realmente importantes em computadores à luz de outras tecnologias avançadas, como

arquiteturas de computador avançadas e tecnologias de fabricação,

interfaces gráficas de usuário (GUIs) fáceis de usar, intuitivas,

sistemas orientados a objetos,

tecnologias integradas da Web e

rede rápida, com e sem fio.

A resposta é sim. Embora alguns aplicativos não exijam explicitamente algoritmo conteúdo de microfone no nível do aplicativo (como alguns aplicativos simples baseados na Web), muitos fazem. Por exemplo, considere um serviço baseado na Web que determina como viajar de um local para outro. Sua implementação dependeria de hardware rápido, um interface gráfica do usuário, rede de área ampla e também, possivelmente, na origem do objeto entação. No entanto, também exigiria algoritmos para certas operações, como como encontrar rotas (provavelmente usando um algoritmo de caminho mais curto), renderizar mapas e endereços de interpolação.

Além disso, mesmo um aplicativo que não requer conteúdo algorítmico no o nível do aplicativo depende muito de algoritmos. O aplicativo depende do rápido hardware? O projeto de hardware usou algoritmos. O aplicativo depende de interfaces gráficas de usuário? O design de qualquer GUI depende de algoritmos. Faz o aplicativo depende de rede? O roteamento em redes depende muito de algoritmos. O aplicativo foi escrito em uma linguagem diferente do código de máquina? Então foi processado por um compilador, intérprete ou montador, todos os quais fazem uso extensivo

de algoritmos. Algoritmos estão no centro da maioria das tecnologias usadas na contemporaneidade computadores diversos.

Além disso, com as capacidades cada vez maiores dos computadores, nós os usamos para resolver problemas maiores do que nunca. Como vimos na comparação acima, entre a classificação de inserção e a classificação de mesclagem, é em tamanhos de problema maiores que as diferenças na eficiência entre algoritmos tornam-se particularmente proeminentes.

Ter uma base sólida de conhecimento e técnica algorítmica é uma característica isso separa os programadores verdadeiramente qualificados dos novatos. Com moderno com colocando tecnologia, você pode realizar algumas tarefas sem saber muito sobre algoritmos, mas com um bom conhecimento em algoritmos, você pode fazer muito, muito Mais.

Exercícios

1,2-1

Dê um exemplo de um aplicativo que requer conteúdo algorítmico no aplicativo nível de ção e discutir a função dos algoritmos envolvidos.

1,2-2

Suponha que estejamos comparando implementações de classificação por inserção e classificação por mesclagem no mesma máquina. Para entradas de tamanho n , a classificação por inserção é executada em $8n^2$ etapas, enquanto a fusão sort é executado em $64n \lg n$ etapas. Para quais valores de n faz a inserção de classificação de batida mesclada ordenar?

1,2-3

Qual é o menor valor de n tal que um algoritmo cujo tempo de execução é $100n^2$ é executado mais rápido do que um algoritmo cujo tempo de execução é 2^n na mesma máquina?

Problemas

1-1 Comparação de tempos de execução

Para cada função $f.n$ / e tempo t na tabela a seguir, determine o maior tamanho n de um problema que pode ser resolvido no tempo t , assumindo que o algoritmo para resolver o problema leva $f.n$ / microssegundos.

	1	1	1	1	1	1
	segundo	minuto	hora	dia	mês	ano
						século
$\lg n$						
P_n						
n						
$n \lg n$						
n^2						
n^3						
2^n						
$n!$						

Notas do capítulo

Existem muitos textos excelentes sobre o tópico geral de algoritmos, incluindo aqueles por Aho, Hopcroft e Ullman [5, 6]; Baase e Van Gelder [28]; Brassard e Bratley [54]; Dasgupta, Papadimitriou e Vazirani [82]; Goodrich e Tamassia [148]; Hofri [175]; Horowitz, Sahni e Rajasekaran [181]; Johnsonbaugh e Schaefer [193]; Kingston [205]; Kleinberg e Tardos [208]; Knuth [209, 210, 211]; Kozen [220]; Levitina [235]; Manber [242]; Mehlhorn [249, 250, 251]; Purdom e Brown [287]; Reingold, Nievergelt e Deo [293]; Sedgewick [306]; Sedgewick e Flajolet [307]; Skiena [318]; e Wilf [356]. Alguns dos mais aspectos práticos do projeto de algoritmos são discutidos por Bentley [42, 43] e Gonnet [145]. Pesquisas no campo dos algoritmos também podem ser encontradas no *Handbook of The-Ciência da Computação teórica, Volume A* [342] e os *CRC Algorithms and Theory Computation Handbook* [25]. Visão geral dos algoritmos usados em computação biologia pode ser encontrada em livros de Gusfield [156], Pevzner [275], Setubal e Meidanis [310] e Waterman [350].